

Competitive Programming

Section 59: ICPC Team Strategy

Competitive Programming Course

2026-07-06

Table of contents

1	Team strategy	3
1.1	Shared contest state	3
1.2	Ownership rules	3
1.3	Keyboard management	3
1.4	Notebook strategy	4
1.5	Communication protocol	4
1.6	Example team board	4
1.7	Notebook discipline	4
1.8	Handoff example	5
1.9	Team debugging roles	5
1.10	Contest communication anti-patterns	5
2	Exercises	6

1 Team strategy

ICPC performance depends on shared state: who owns which problem, when to switch, how to preserve debugging context, and how to maintain a notebook. The algorithmic drills here are small but represent team operations: ranking, balancing, and choosing known templates quickly.

1.1 Shared contest state

The team needs a visible board:

Problem	owner	idea	status	blocker	next action
---------	-------	------	--------	---------	-------------

Without this, two people duplicate work or a promising partial idea gets lost during a switch.

1.2 Ownership rules

Each active problem should have one owner. Other teammates can review, test, or derive math, but the owner decides when to code and submit. This prevents half-merged ideas.

Switch ownership when:

- the owner is stuck after a fixed time box;
- a teammate sees a complete solution;
- the problem needs a template another teammate knows better.

1.3 Keyboard management

For one-machine ICPC, the keyboard is a scarce resource. Good teams separate:

- thinking on paper;
- writing tests;
- coding final solutions;
- debugging.

Do not let one long debugging session block all implementation unless the problem is clearly near accepted.

1.4 Notebook strategy

A good notebook is short and trusted. Include:

- DSU, Fenwick, segment tree;
- Dijkstra, Dinic, SCC/2-SAT;
- geometry primitives;
- KMP/Z/hash;
- CRT, modular inverse, matrix or Fibonacci doubling.

Every template should have a tiny usage note and known indexing convention.

1.5 Communication protocol

When handing off a problem, say:

```
I think the model is ...  
The invariant is ...  
The failing case is ...  
The next thing to try is ...
```

This is faster than retelling the whole statement.

1.6 Example team board

Problem	Owner	Status	Next action
A	Ana	AC	none
B	Ben	coding	test overflow
C	Chen	idea found	wait for keyboard
D	Ana	reading	decide model
E	none	untouched	scan later

This board prevents silent duplication. It also makes it easier to interrupt work: if Ben leaves the keyboard, the next person knows the current risk.

1.7 Notebook discipline

Do not include a template you do not trust. A shorter notebook with tested code beats a long notebook full of half-remembered snippets. Every notebook template should have:

```
input convention  
indexing convention  
complexity  
one tiny sample use
```

1.8 Handoff example

Bad handoff:

```
"I tried C, it is weird."
```

Good handoff:

```
"C looks like bipartite matching. Left side is tasks, right side is people.  
The bug is that person capacity is 2, so we either duplicate person nodes or use  
flow. I tested the sample graph by hand; max answer should be 4."
```

The second handoff preserves the model, the blocker, and the next action.

1.9 Team debugging roles

When one person codes, the reviewer should not just watch syntax. The reviewer tracks:

```
indexing convention  
overflow risk  
sample walkthrough  
unhandled edge cases  
whether the code still matches the proof
```

This makes debugging parallel even with one keyboard.

1.10 Contest communication anti-patterns

Avoid:

```
"It should work."  
"The bug is somewhere in the code."  
"This is probably DP."
```

Prefer:

```
"The invariant is prefix parity."  
"The failing case is n=1."  
"The DP state lacks the previous color."
```

Concrete statements let teammates act.

2 Exercises

- A. Team Assignment: Split implementation workload between two people minimizing max time.
- B. Team Scoreboard: Rank teams by contest rules.
- C. Template Fibonacci: Fast recurrence query drill for the team notebook. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.